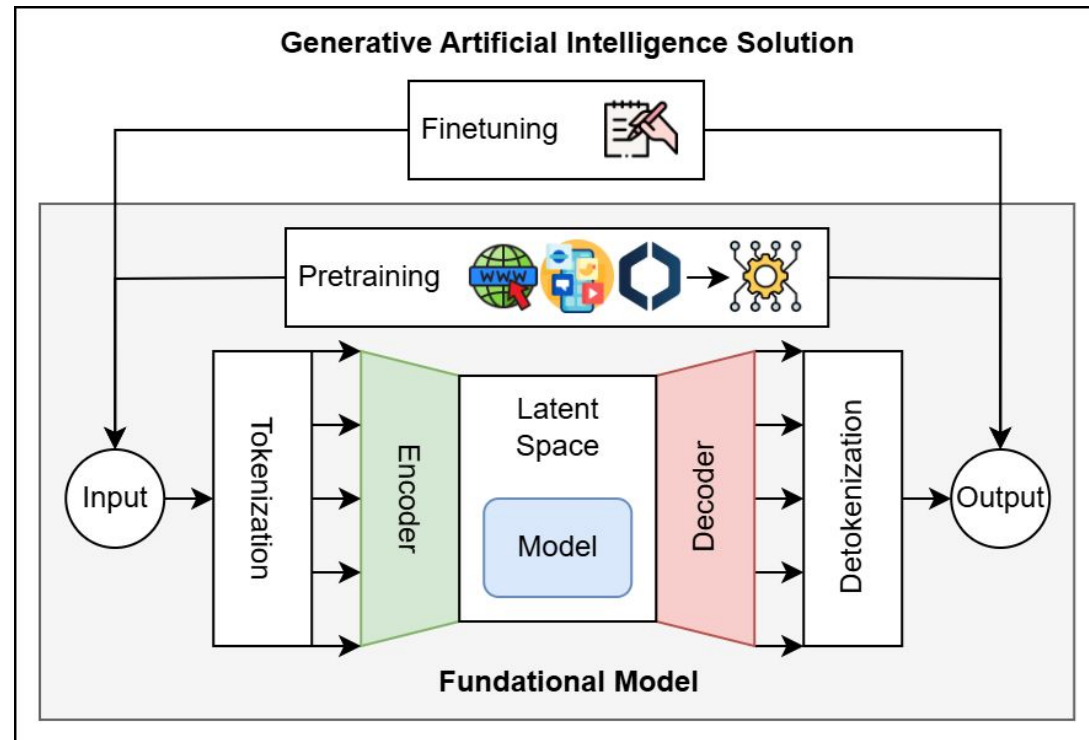


Modelos Generativos de Imagem

Thiago Brandenburg
2025/1

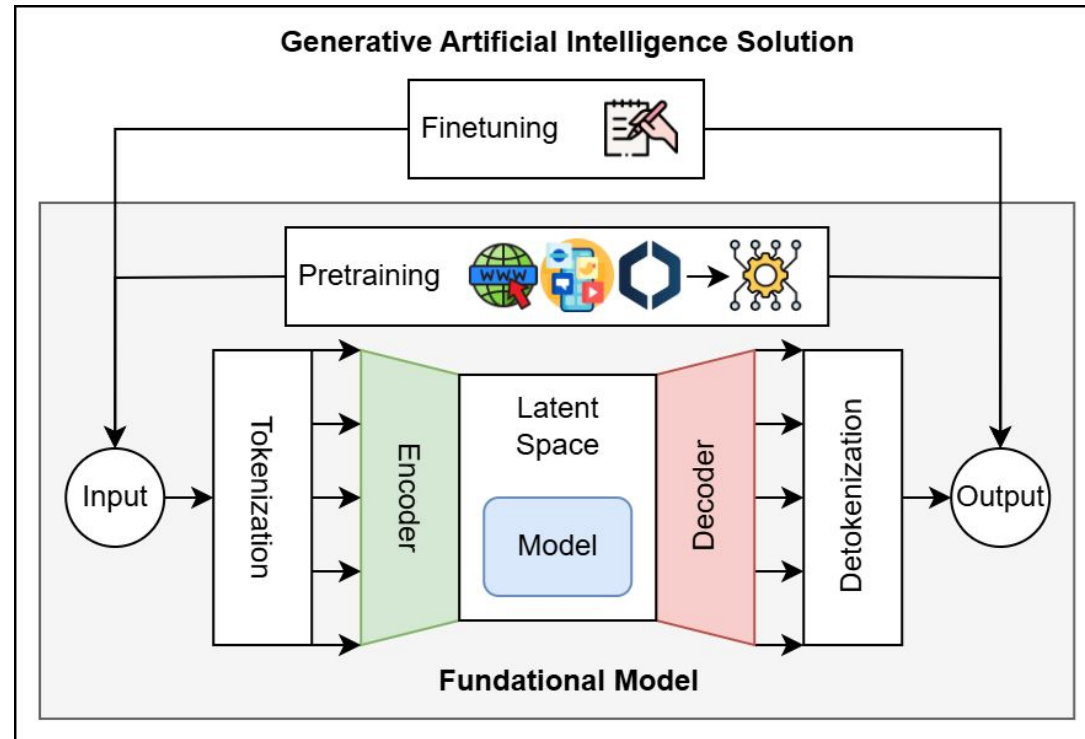
Revisão:

- **Partes do Modelo:**
 - Tokenização
 - Embedding
 - Arquitetura do modelo.
- **Treinamento em Duas etapas:**
 -



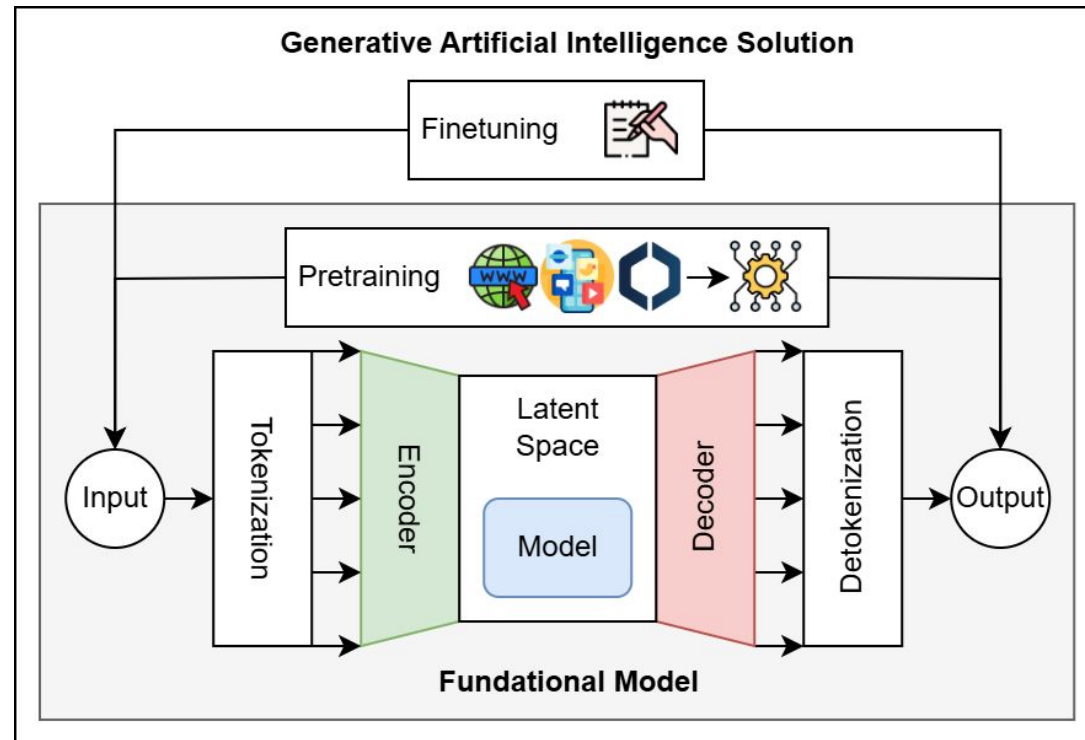
Revisão:

- **Partes do Modelo:**
 - Tokenização
 - Embedding
 - Arquitetura do modelo.
- **Treinamento em Duas etapas:**
 - **Pre-training:** Grande dataset, não estruturado, dados da internet, gera **modelo fundacional**.
 -



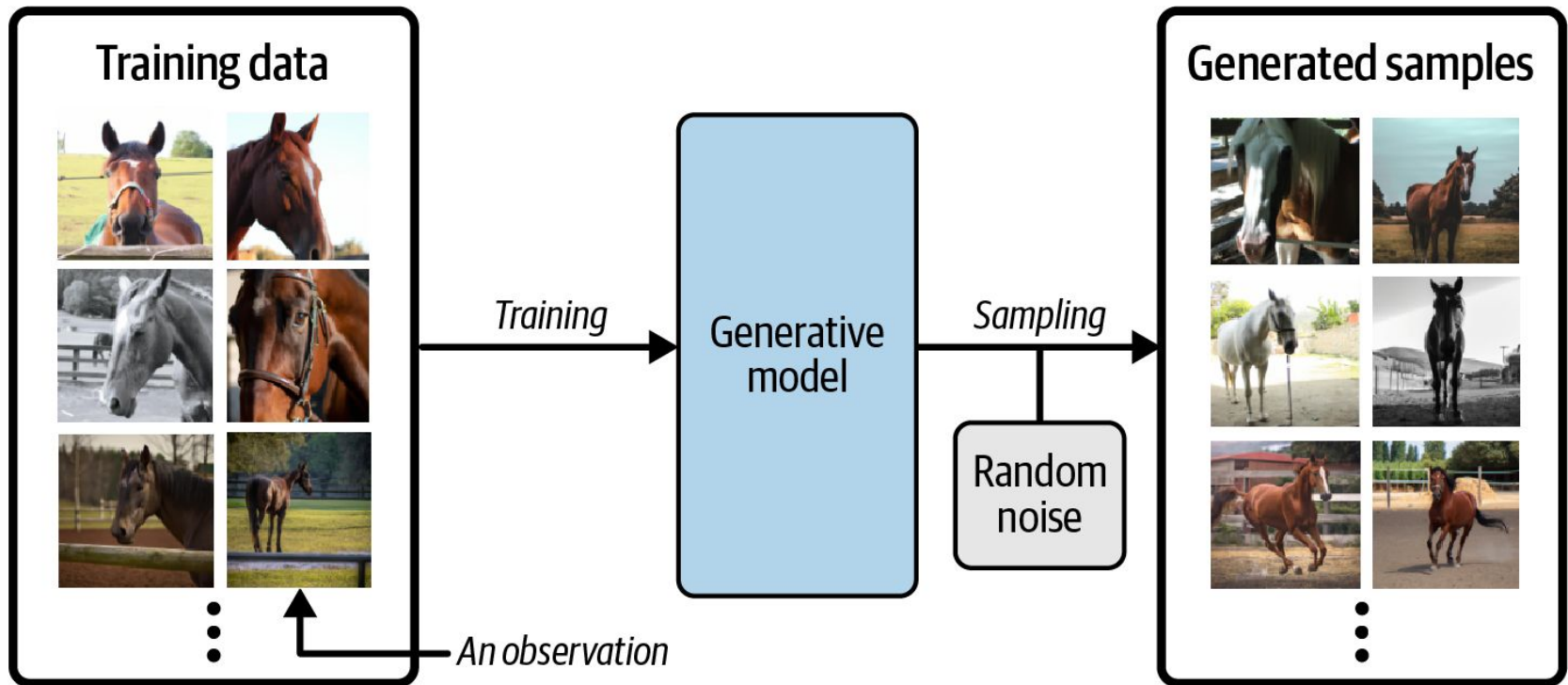
Revisão:

- **Partes do Modelo:**
 - Tokenização
 - Embedding
 - Arquitetura do modelo.
- **Treinamento em Duas etapas:**
 - **Pre-training:** Grande dataset, não estruturado, dados da internet, gera **modelo fundacional**.
 - **Fine-tuning:** Datasets menores, dados rotulados, funções especializadas, modelos otimizados (quantização e LoRA), treinamento pelo usuário.



Modelos de Imagem: Definição

- Modelos generativos de Imagem geram e/ou manipulam imagens, buscando criar imagens o mais coerentes possíveis com o contexto.



Modelos de Imagem: Características

- **Processamento em GPU:** Imagens são pesadas demais para a CPU!
 - Quantos parâmetros temos que processar para 1 única imagem em FULL HD?

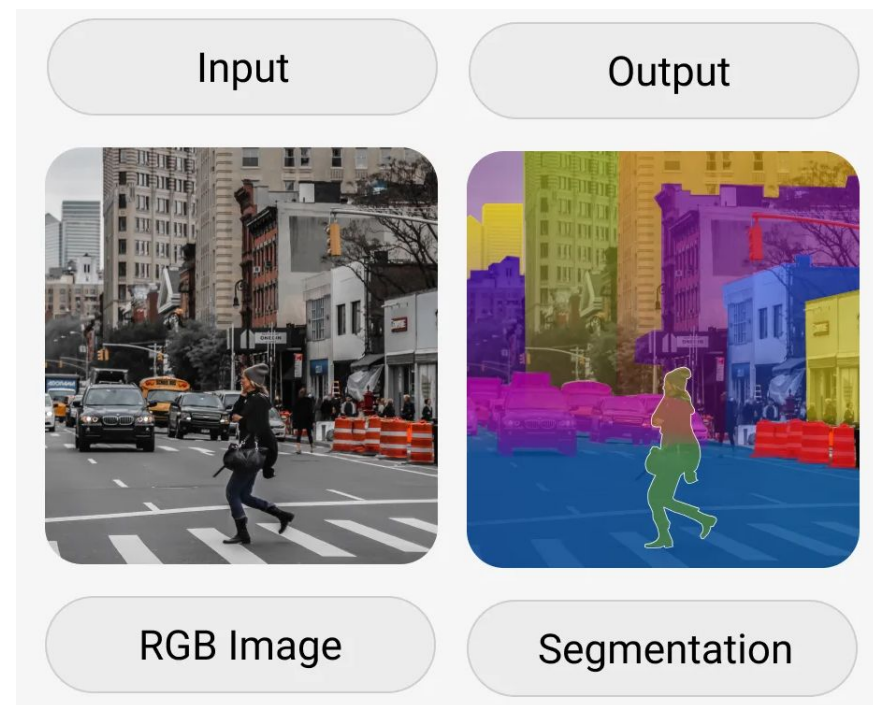


Modelos de Imagem: Características

- Processamento em GPU: Imagens são pesadas demais para a CPU!
- Tokenização em Patches:
 - **Hard Tokens:** Separação da Imagem em partes.
 - **Soft Tokens:** Convoluções.



Hard Tokens



Soft Tokens

Modelos de Imagem: Características

- Processamento em GPU: Imagens são pesadas demais para a CPU!
- Tokenização em Patches:
 - Hard Tokens: Separação da Imagem em partes.
 - Soft Tokens: Convoluções
- **Multimodais:**
 - Image-to-Image.
 - **Image-to-Text**
 - **Text-to-Image**
- É preciso colocar **Imagens e Textos** em uma mesma linguagem:



“an astronaut riding
a horse”



Modelos de Imagem: Características

- Processamento em GPU: Imagens são pesadas demais para a CPU!
- Tokenização em Patches:
 - Hard Tokens: Separação da Imagem em partes.
 - Soft Tokens: Convoluções
- Multimodais:
 - Image-to-Image.
 - **Image-to-Text**
 - **Text-to-Image**
- É preciso colocar **Imagens e Textos** em uma mesma linguagem:
 - **Embeddings!**

“an astronaut riding
a horse”



■ Embeddings de Imagem

- Coloque-se no lugar do modelo generativo:



Embeddings de Imagem

- Coloque-se no lugar do modelo generativo:
 - Temos dois problemas:
 - i. **Problema 1:** Como criar embeddings de Imagem?
 - ii. **Problema 2:** Como criar correspondência entre Embeddings de Imagens e Texto?



Embeddings de Imagem

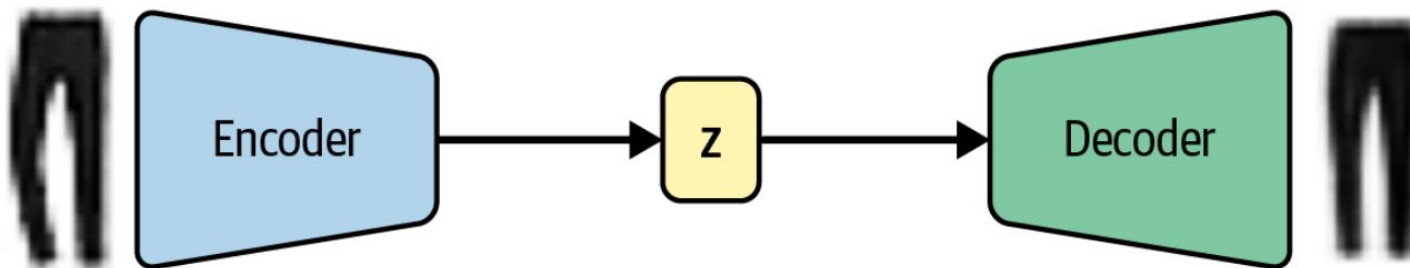
1. Como criar embeddings de Imagem?



Embeddings de Imagem

1. Como criar embeddings de Imagem?

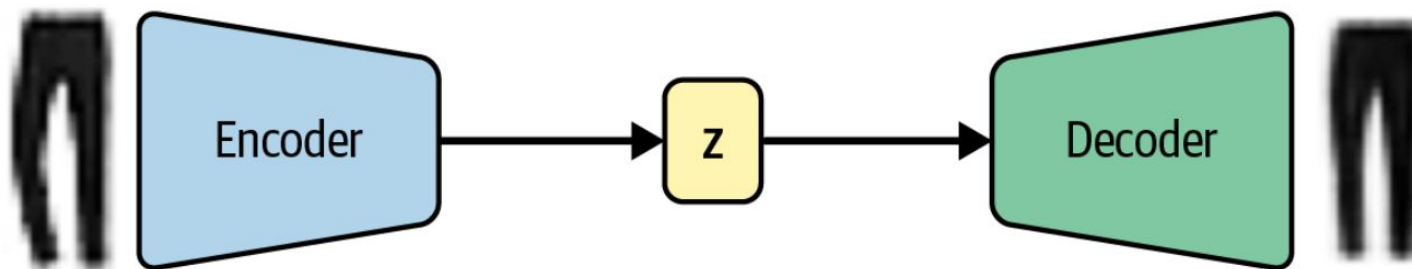
- Uma arquitetura Encoder/Decoder Clássica como em uma LLM funciona?



Embeddings de Imagem

1. Como criar embeddings de Imagem?

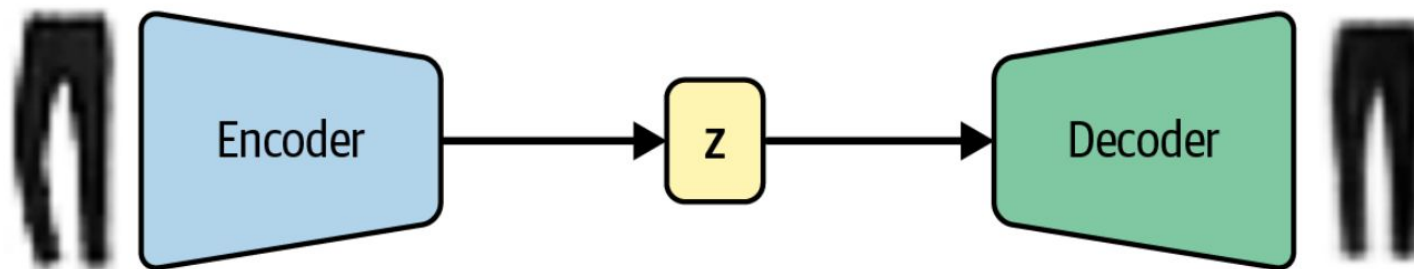
- Uma arquitetura Encoder/Decoder Clássica como em uma LLM funciona? **Sim!**
 - “Calça” terá uma posição no espaço latente.
 - Se adicionamos um ruído na posição de calça?



Embeddings de Imagem

1. Como criar embeddings de Imagem?

- Uma arquitetura Encoder/Decoder Clássica como em uma LLM funciona? **Sim!**
 - “Calça” terá uma posição no espaço latente.
 - Se adicionamos um ruído na posição de calça? **Diferentes calças!**



Embeddings de Imagem

1. Como criar embeddings de Imagem?

- Uma arquitetura Encoder/Decoder Clássica como em uma LLM funciona? **Sim!**
 - “Calça” terá uma posição no espaço latente.
 - Se adicionamos um ruído na posição de calça? **Diferentes calças!**
 - **Encoder/Decoders são modelos generativos de imagens por si só!**
 - Diferentes variações da arquitetura são utilizadas:
 - U-nets
 - Variational Autoencoders (VAEs)
 - Redes Neurais Adversárias com Encoder/Decoders

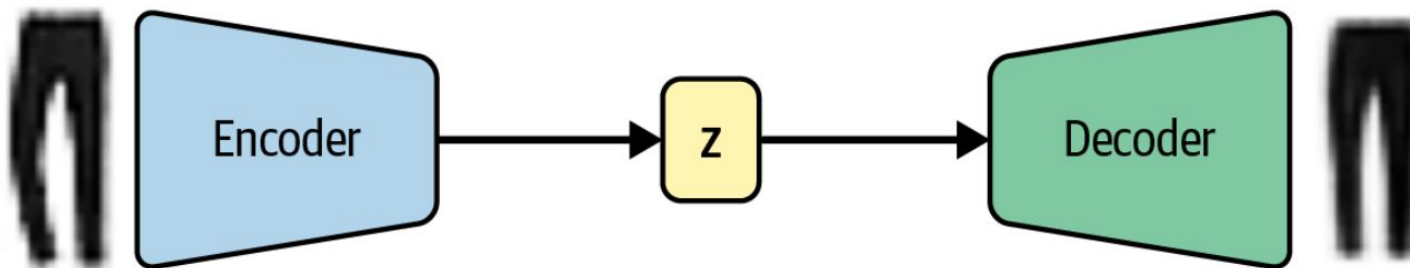
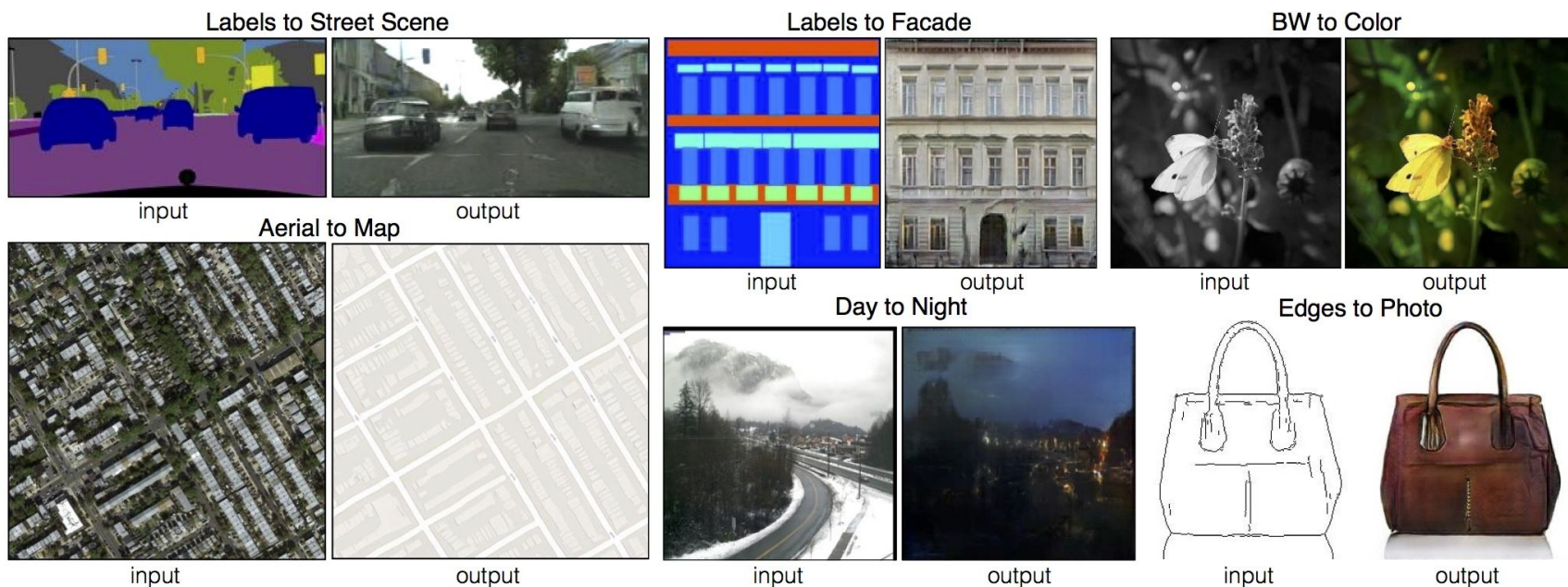


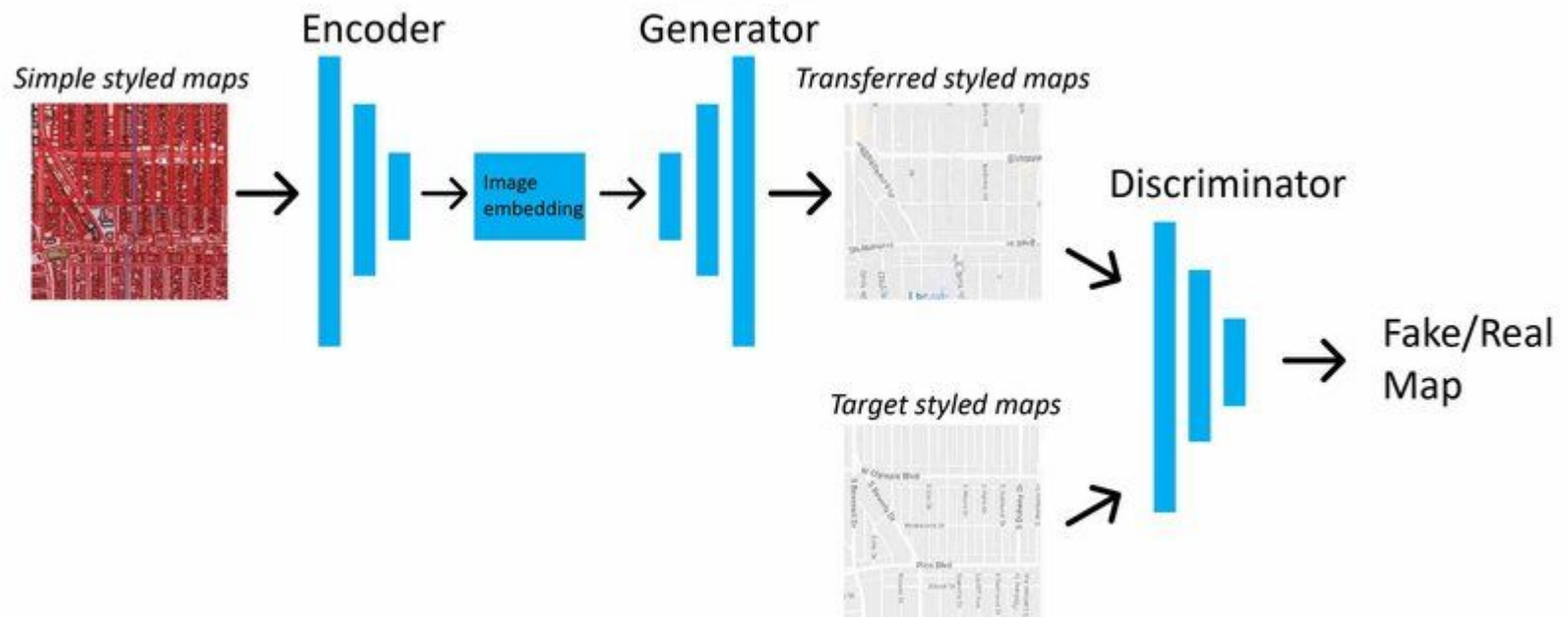
Image-to-Image

- Resolvendo a primeira questão, podemos criar modelos que alteram imagens!
- Exemplos: Super-resolução, coloração, pós processamento, etc.
- O modelo **Pix2Pix** é o mais popular nessa categoria, e utiliza uma **Rede Neural Adversarial (RNA)**.



Pix2Pix: Redes Adversárias Generativas

- Dois modelos:
 - **Discriminador (D)**: Um rede neural profunda treinada para dizer se uma entrada é real ou gerada por modelo.
 - **Gerador (G)**: Um modelo que gera amostras. Pix2Pix usa ou Autoencoders ou U-Nets
- Ajustamos G para “enganar” D.



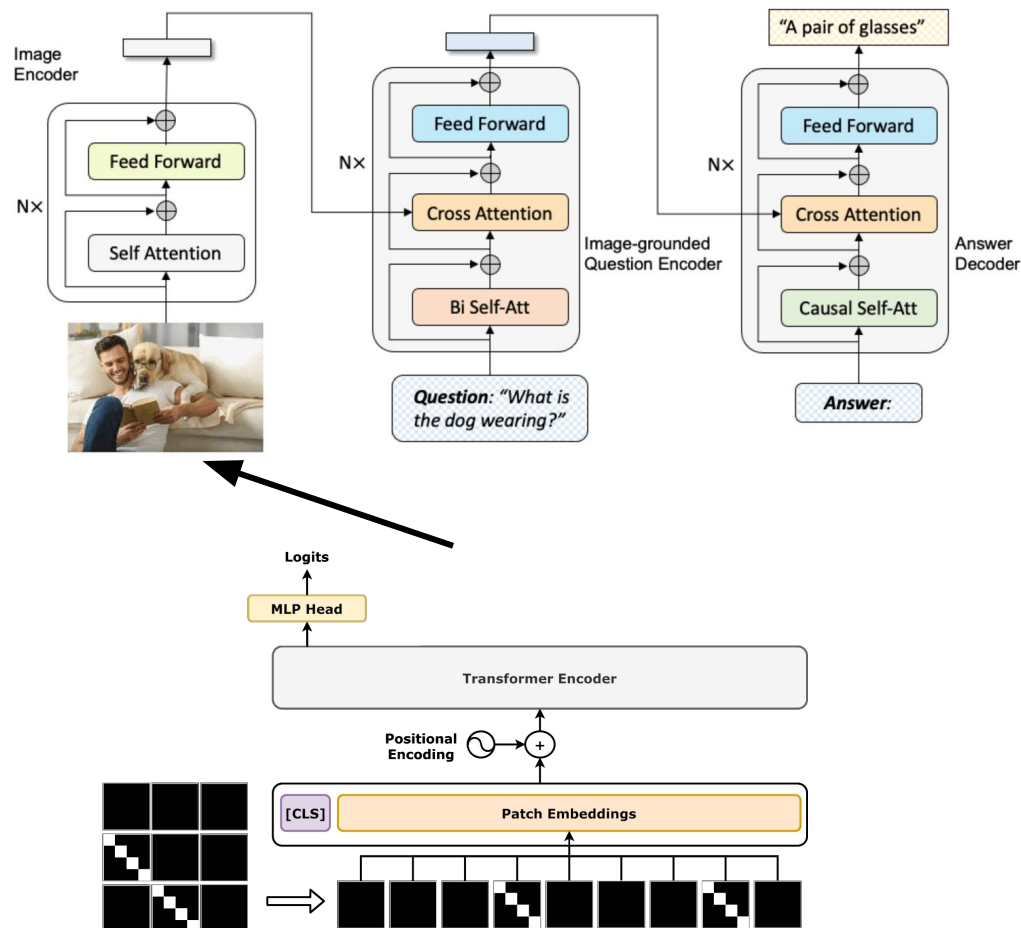
Modelos de Imagem: Embeddings

2. Como criar correspondência entre Embeddings de Imagens e Texto?

- Precisamos de Encoders!
 - Se queremos um LLM que descreva imagens (image-to-text), precisamos de um Encoder de Imagem para um espaço latente compreensível por um LLM.
 - Se queremos um modelo que gera imagens a partir de um prompt (text-to-image), precisamos de um Encoder de Texto para um espaço latente compreensível por um modelo de Imagem.
- Soluções:
 - **Blip**: Treina de forma eficiente um encoder de imagem para um decoder de texto (LLMs).
 - **Clip**: Treina pares de encoder/decoder projetando ambos para o mesmo espaço vetorial.
 - Dentre outros métodos para treinar encoders.

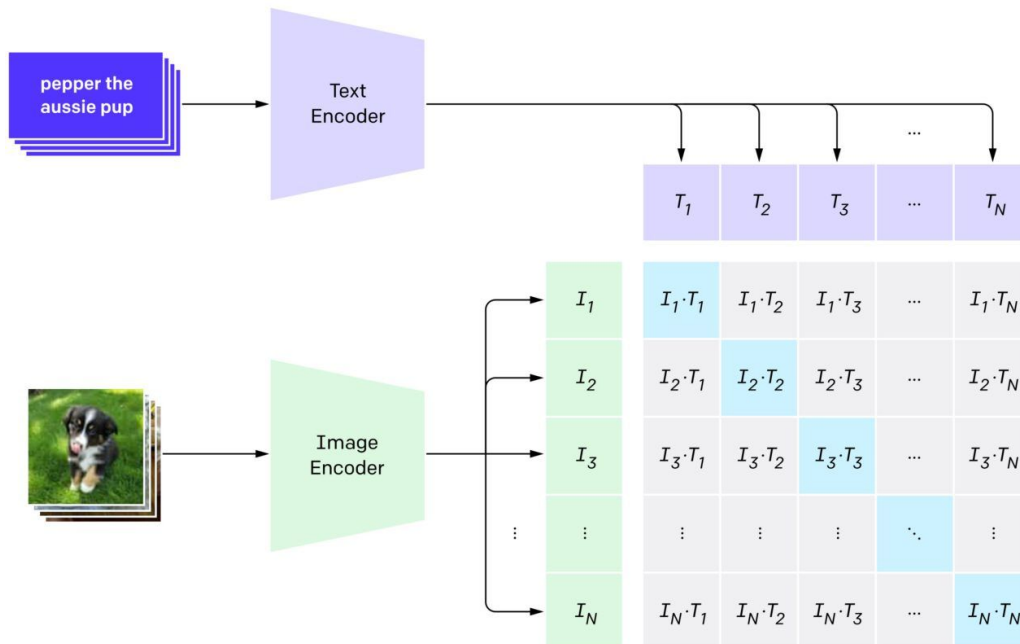
BLIPs Bootstrapping Language-Image Pre-training

- Compostos em três partes:
- **Image Encoder:** Um Vision Transformer (ViT) que transforma a imagem em hard tokens.
- **Question Encoder:** Transformer Encoder que atua como modelo intermediário. Une pergunta e Imagem em um único embedding e envia para o Answer Encoding.
- **Answer Decoder:** Um LLM (Transformer Decoder)
- Acoplam interpretabilidade de Imagem (Image-to-Text) em LLMs.



CLIP (Contrastive Language-Image Pre-training)

1. Cada batch de treino terá n pares de imagens e rótulos;
2. Criamos dois **Encoders**: Um para **imagem** e um para o **texto**.
3. Criamos uma matriz com o produto escalar dos embeddings gerados por cada encoder.
4. Maximizamos a diagonal principal e minimizamos as matrizes triangulares superior e inferior
 - a. symmetric contrastive loss function



Atenção!

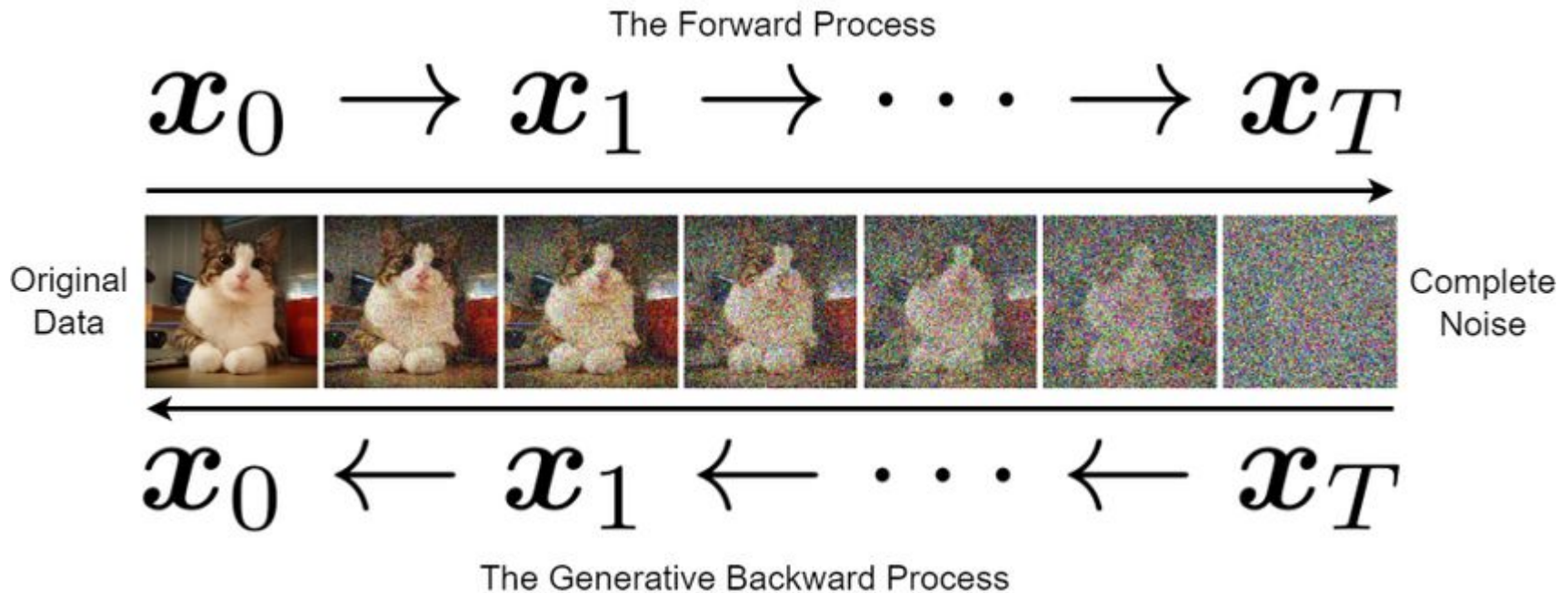
- O Clip **não** mapeia diretamente para o espaço latente do modelo de destino.
- ele apenas cria embeddings fáceis de interpretar pelo modelo de destino durante o treinamento.

O que sabemos até agora

- Arquiteturas Encoder/Decoder são usadas para gerar imagens.
- Modelos Image-to-Image:
 - Alteram Imagens.
 - Pix2Pix é o mais popular.
 - Utilizam redes adversárias generativas (GAN).
- Modelos Image-to-Text são LLMs com encoder de imagem para texto embutidos.
- **E os modelos text-to-image?**

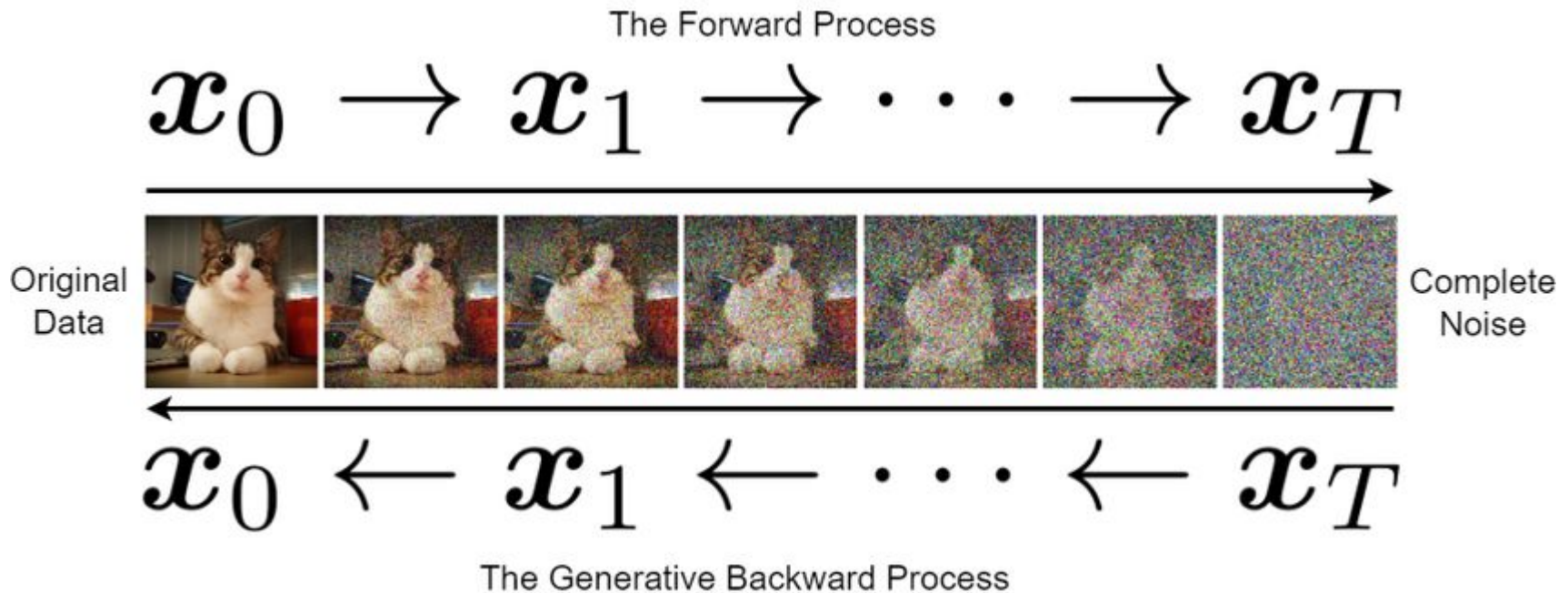
Modelos Text-to-Image

- Quando se referimos a modelos generativos de imagens,
- Atualmente são utilizados modelos de difusão para geração de imagens;
 - Modelos treinados para tirar ruídos de imagem iterativamente.
 - Como fazemos para gerar uma imagem nova com esse modelo?



Modelos Text-to-Image

- Quando se referimos a modelos generativos de imagens,
- Atualmente são utilizados modelos de difusão para geração de imagens;
 - Modelos treinados para tirar ruídos de imagem iterativamente.
 - Como fazemos para gerar uma imagem nova com esse modelo?
 - Retiramos ruído de uma imagem aleatória

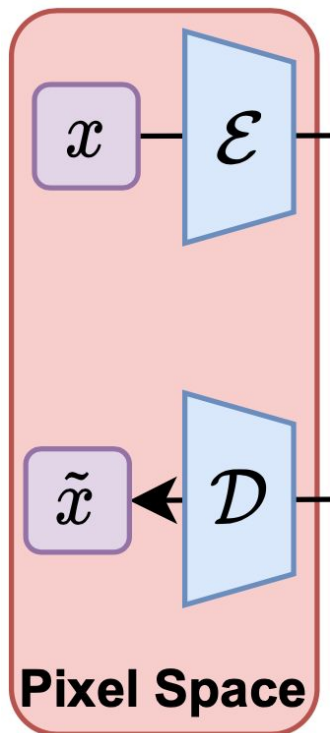


Modelos de Difusão: Stable Diffusion

- Vamos explicar como funciona um modelo de difusão utilizando o Stable Diffusion (SD) como exemplo.
 - Open Source.
 - Arquitetura ainda é utilizada por outros modelos mais avançados.
 - Diagrama didático.

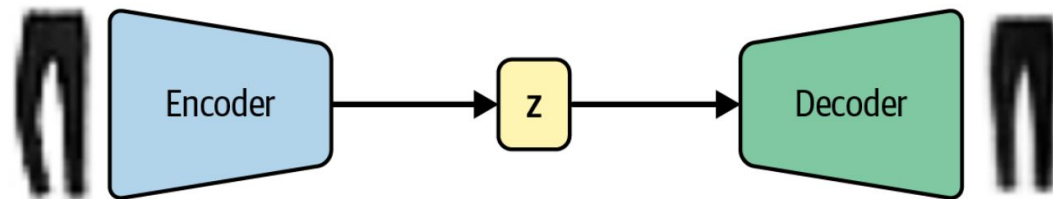
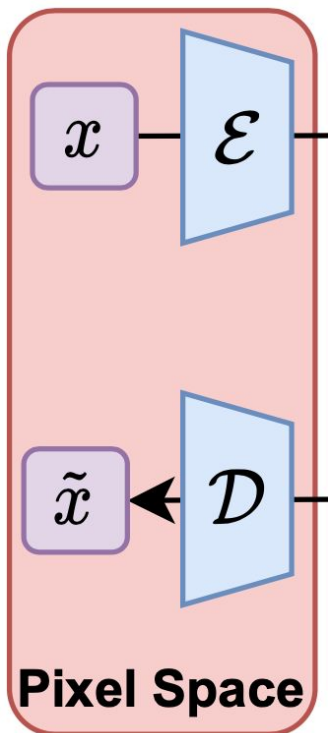
Modelos de Difusão: Stable Diffusion

- A Primeira camada de um modelo de difusão é o Encoder/Decoder.
- Geralmente é utilizado um **Autoencoder**, que apesar de não ser um modelo generativo robusto, é muito eficiente em criar o **espaço latente**.
- Modelos de difusão usam **soft tokens**: não há uma tokenização direta, apenas convoluções, os pixels viram patches pelo encoder.



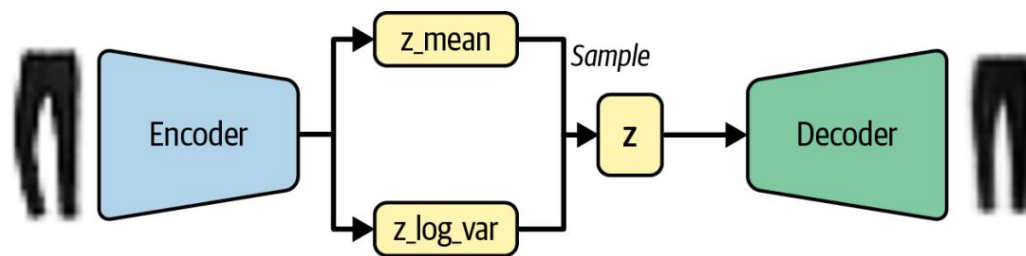
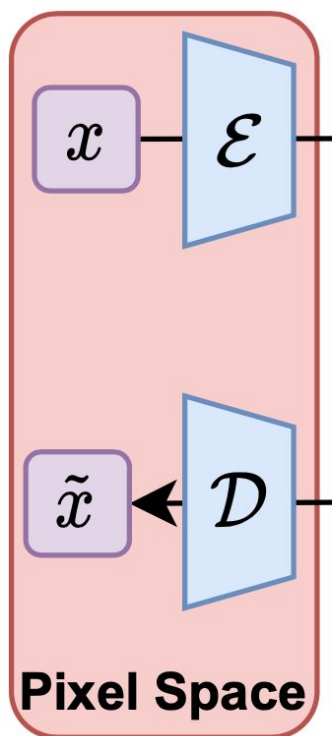
Modelos de Difusão: Stable Diffusion

- Existe um problema no autoencoder: ele é determinístico.
- Não temos o luxo de uma saída com uma função de probabilidade.
 - Como resolver?



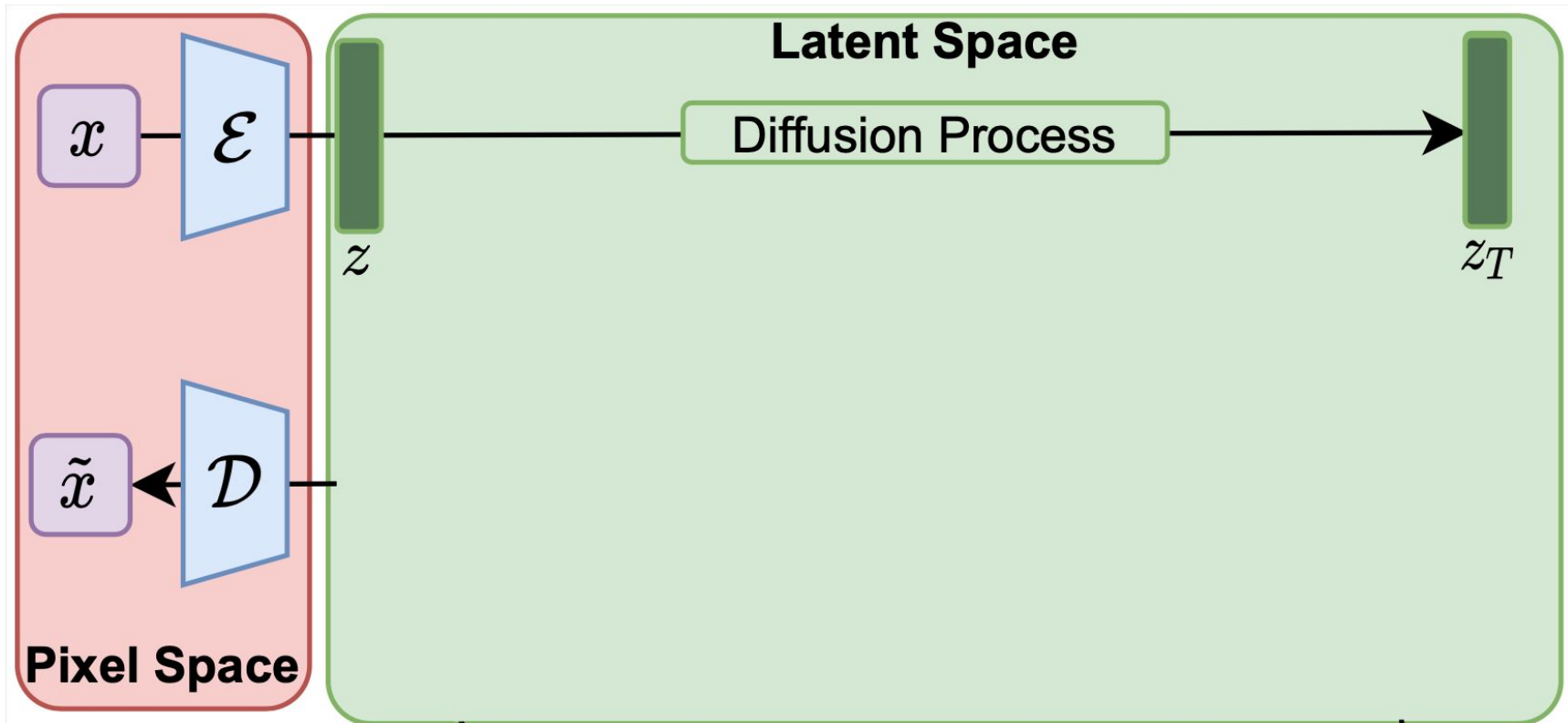
Modelos de Difusão: Stable Diffusion

- Ao invés dos embeddings serem 1 único vetor (ponto), mapeamos para dois vetores que utilizamos para construir a função de probabilidade!
- Embedding = média + log(variância) * $N(0,1)$
- Essa arquitetura estocástica se chama **Variational Autoencoder**
- O resto do modelo é construído



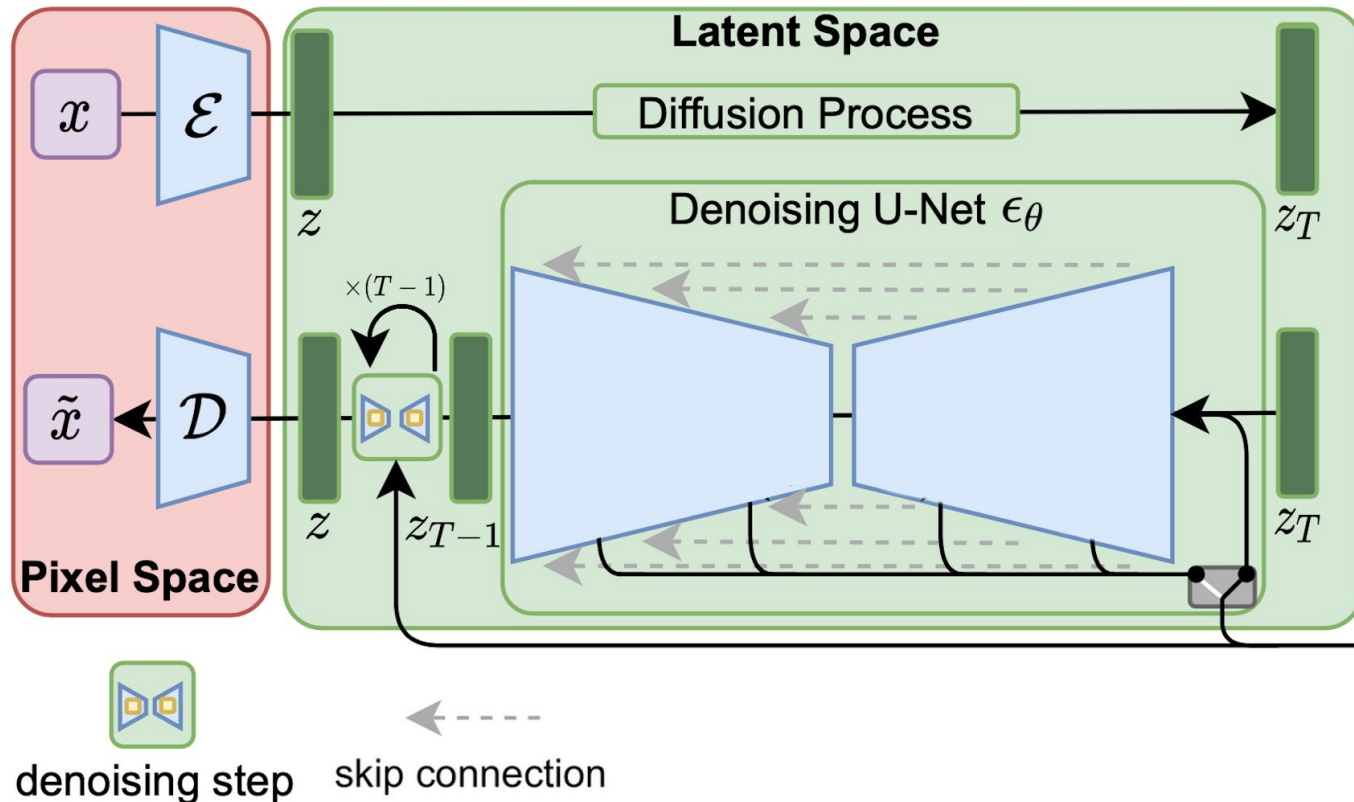
Modelos de Difusão: Stable Diffusion

- Dentro do espaço latente, definimos T níveis de ruído.
- Onde Z_0 é a imagem original, e Z_t a imagem com T níveis de ruído (aleatória).



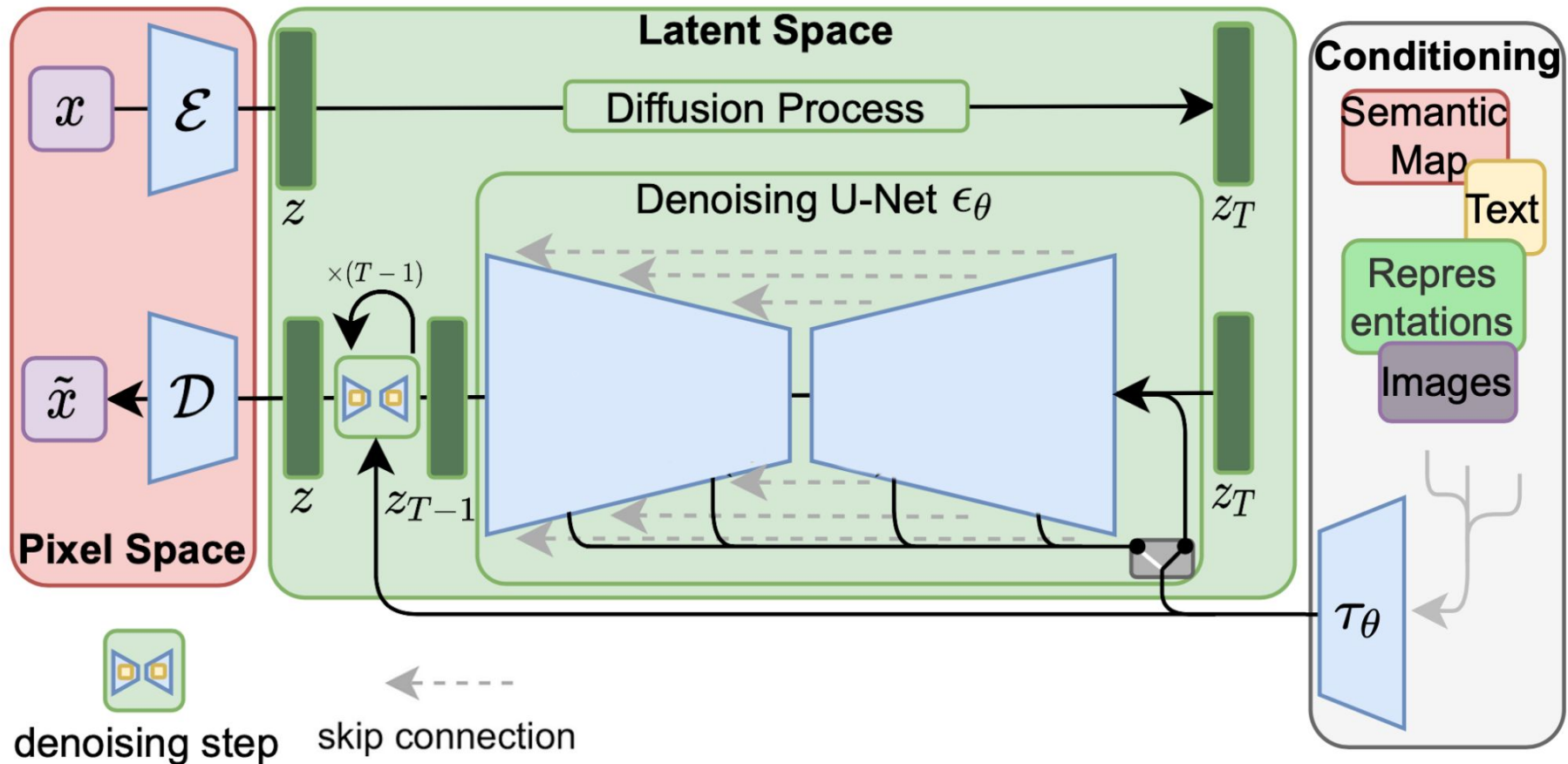
Modelos de Difusão: Stable Diffusion

- Utilizamos uma U-net (Encoder/Decoder com conexões que evitam uma reconstrução total da imagem) para retirar o ruído da imagem.
- Cada ciclo retira um nível de ruído, até obtermos uma imagem sem ruído.



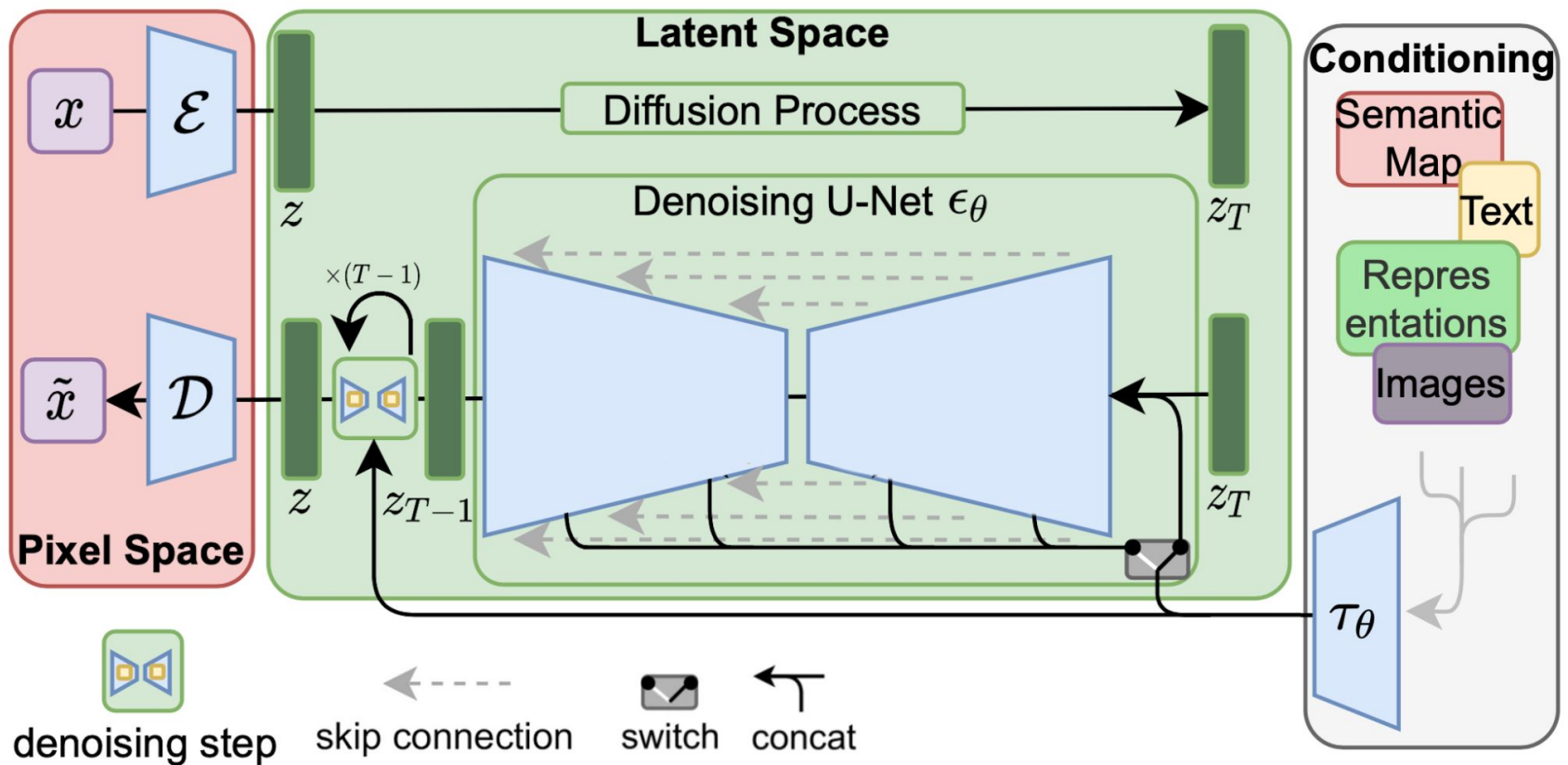
Modelos de Difusão: Stable Diffusion

- Utilizamos o **CLIP** de texto para passar uma descrição da imagem a ser gerada a cada etapa de remoção de ruído.



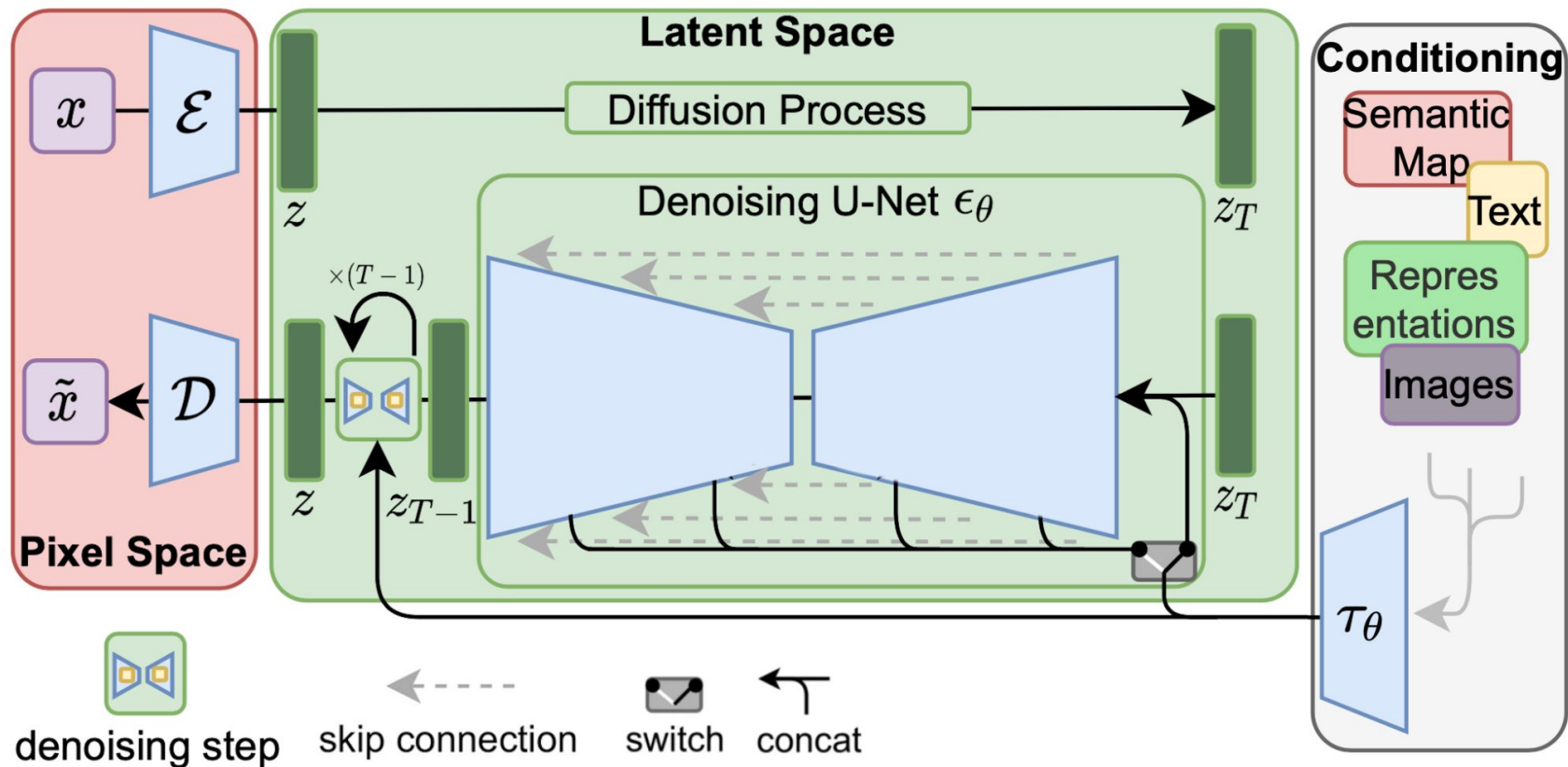
Modelos de Difusão: Stable Diffusion

- O SD ainda utiliza um Switch para decidir quais patches (partes da imagem) vão ter acesso ao contexto.



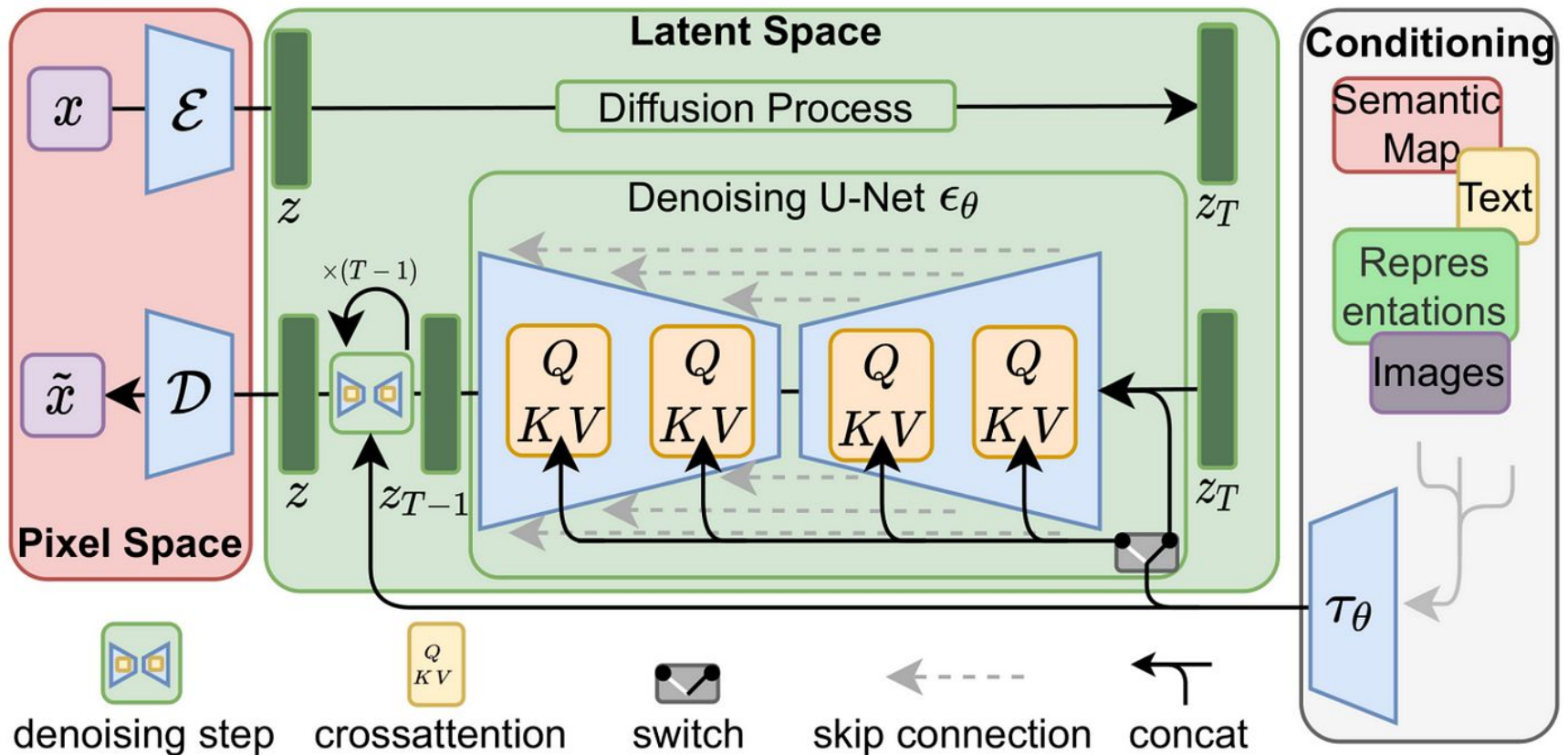
Modelos de Difusão: Stable Diffusion

- A última etapa é garantir que os patches gerados pelas convoluções estejam atentos as outras partes da imagem e ao contexto para retirarmos ruído?
 - Vocês conhecem alguma estrutura que faz isso?



Modelos de Difusão: Stable Diffusion

- A última etapa é garantir que os patches gerados pelas convoluções estejam atentos as outras partes da imagem e ao contexto para retirarmos ruído?
 - Vocês conhecem alguma estrutura que faz isso? **Attention heads.**



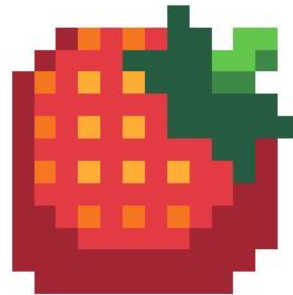
Vídeo Para Visualização: How stable diffusion & dall-e work

- Deixo o vídeo do Computerphille:
[How AI Image Generators Work \(Stable Diffusion / Dall-E\)](#)
- Existe ainda mais uma etapa não apresentada que otimiza o output do modelo. Qual?
 - Terá um Exercício no questionário.



Códigos para Análise

- Deixo o tutorial de modelos generativos de imagem da PyBr 2023:

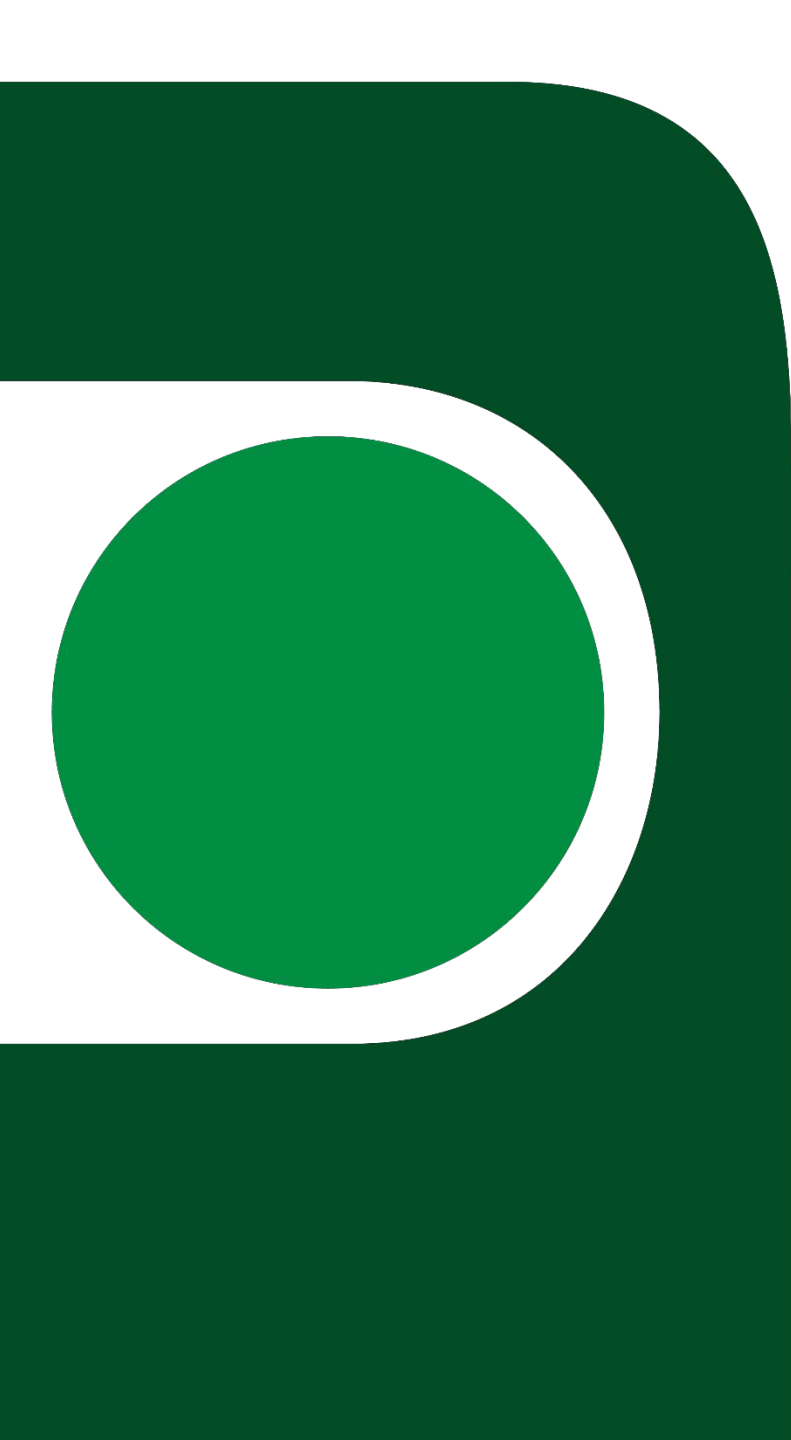


Referências

- FOSTER, David. **Generative deep learning**. " O'Reilly Media, Inc.", 2022.
- HUGGINGFACE. **Image-to-text**. Disponível em: <https://huggingface.co/tasks/image-to-text>. Acesso em: 5 jun. 2025.
- ISOLA, Phillip et al. **Image-to-Image Translation with Conditional Adversarial Networks**. 2017. Disponível em: <https://arxiv.org/pdf/1611.07004>. Acesso em: 5 jun. 2025.
- PHILLIPI. **pix2pix GitHub repository**. Disponível em: <https://github.com/phillipi/pix2pix?tab=readme-ov-file>. Acesso em: 5 jun. 2025.
- **Transferring Multiscale Map Styles Using Generative Adversarial Networks**. Disponível em: https://www.researchgate.net/publication/332932603_Transferring_Multiscale_Map_Styles_Using_Generative_Adversarial_Networks. Acesso em: 5 jun. 2025.
- RODRIGUES, Acucena. **Geração de imagens com Pix2Pix**. Disponível em: https://github.com/acucenarodrigues1998/TutorialPyBR2023-GenAINotebooks/blob/main/Tutorial_PyBR_02_Gera%C3%A7%C3%A3o_de_imagens_com_Pix2Pix.ipynb. Acesso em: 5 jun. 2025.
- **BLIP model documentation**. Disponível em: https://huggingface.co/docs/transformers/model_doc/blip. Acesso em: 5 jun. 2025.

Referências

- RADFORD, Alec et al. **Learning Transferable Visual Models From Natural Language Supervision**. OpenAI, 2021. Disponível em: <https://arxiv.org/pdf/2204.06125>. Acesso em: 5 jun. 2025.
- COMPUTERPHILE. **CLIP - The AI That Understands Text and Images**. YouTube. Disponível em: <https://youtu.be/KcSXCpluDe4>. Acesso em: 5 jun. 2025.
- OPENAI. **CLIP**. Disponível em: <https://openai.com/index/clip/>. Acesso em: 5 jun. 2025.
- ROMBACH, Robin et al. **High-Resolution Image Synthesis with Latent Diffusion Models**. 2021. Disponível em: <https://arxiv.org/abs/2112.10752>. Acesso em: 5 jun. 2025.



Obrigado

**UDESC – Universidade do Estado de
Santa Catarina**

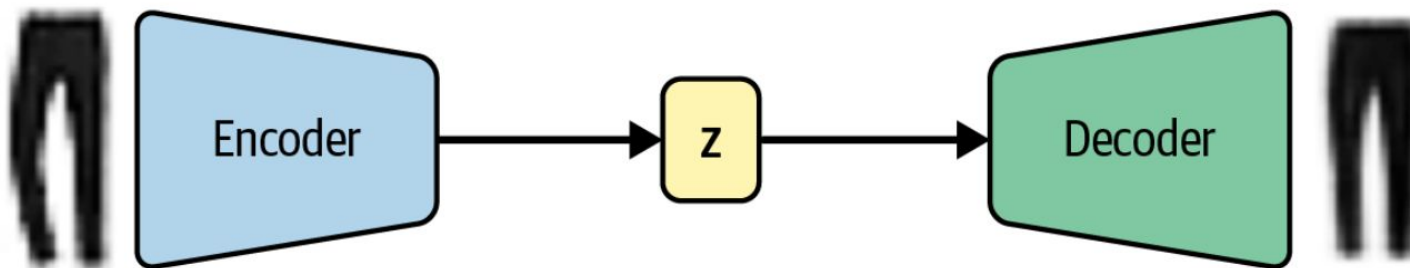
Thiago Brandenburg

thiago.brandenburg@edu.udesc.br

Embeddings de Imagem

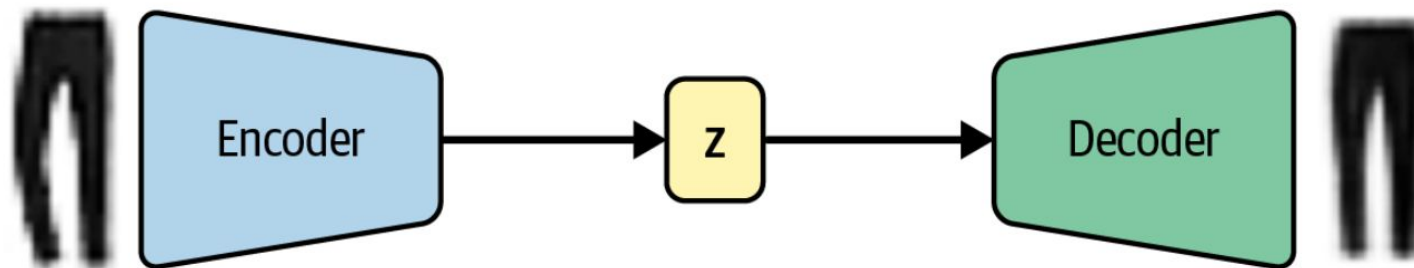
1. Como criar embeddings de Imagem?

- Uma arquitetura Encoder/Decoder Clássica como em uma LLM funciona? **Sim!**
 - “Calça” terá uma posição no espaço latente.
 - Adicionamos um pequeno ruído na posição de calça e obteremos diferentes calças!
 - Basta pegarmos a posição da calça primordial, a calça original, a calça das calças, variarmos um pouquinho cuidando para não esbarrar nos embeddings das bermudas, das saias, das meias calças, etc e tá tudo resolvido.
 - Será que funciona mesmo...?



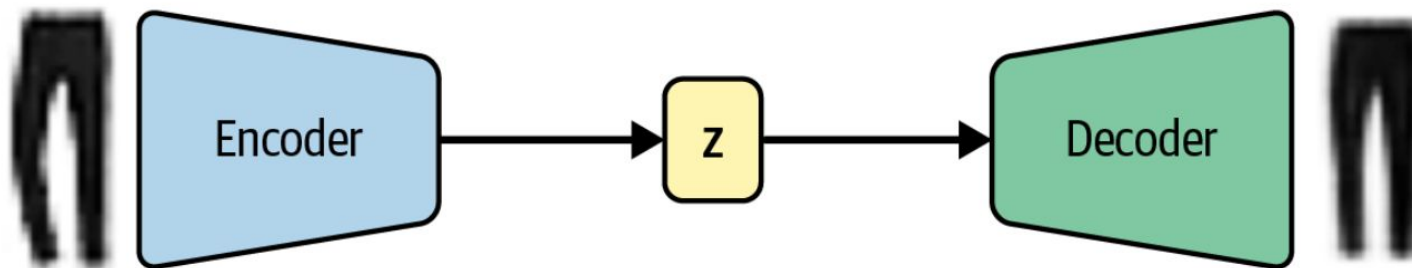
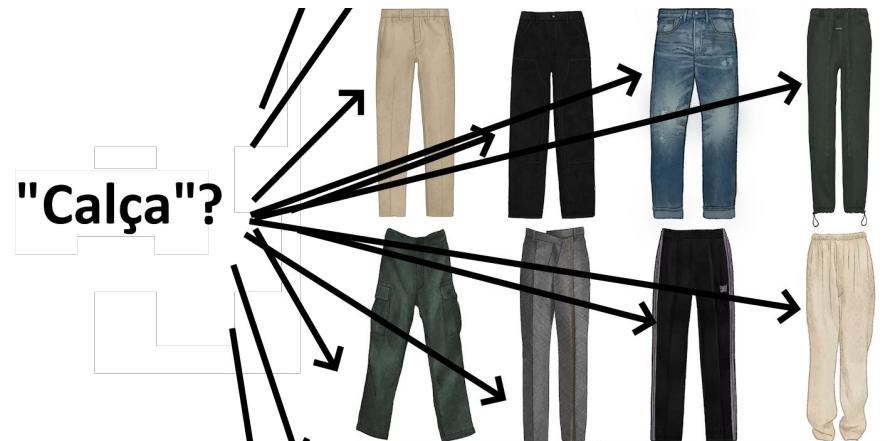
Embeddings de Imagem

- **Problema:** Embeddings em um Encoder/Decoder são **determinísticos!**
- Funciona para palavras, todas as calças são mapeadas para um único embedding de “calça”.
- Para representar diferentes calças, basta descrevê-las com uma sequência de embeddings.



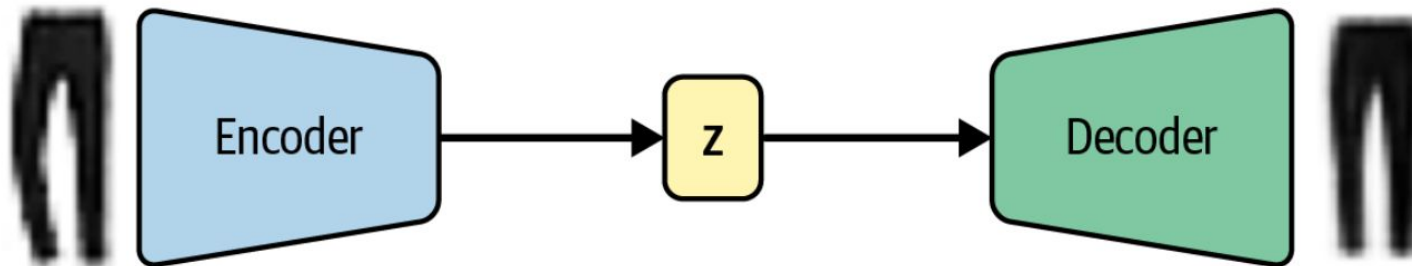
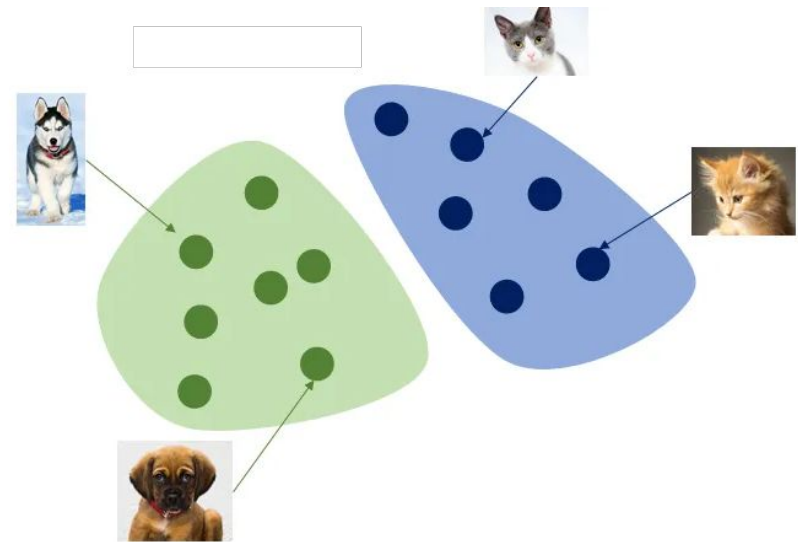
Embeddings de Imagem

- Não temos esse luxo para imagens.
- Imagens completamente diferentes podem ser uma “calça”.



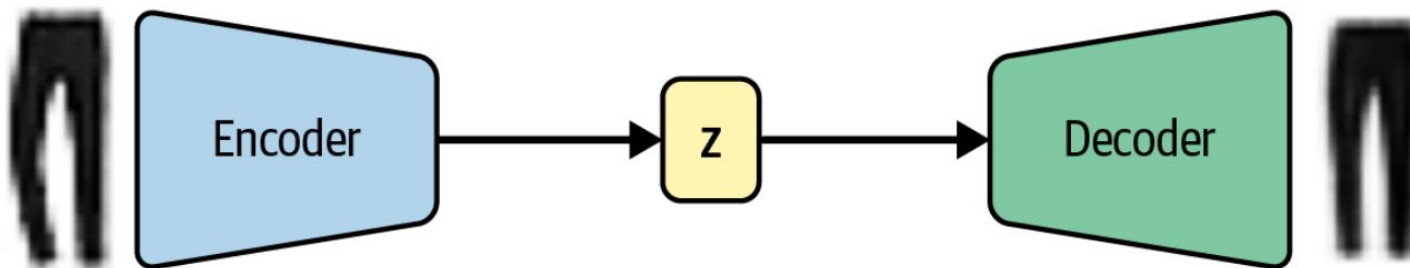
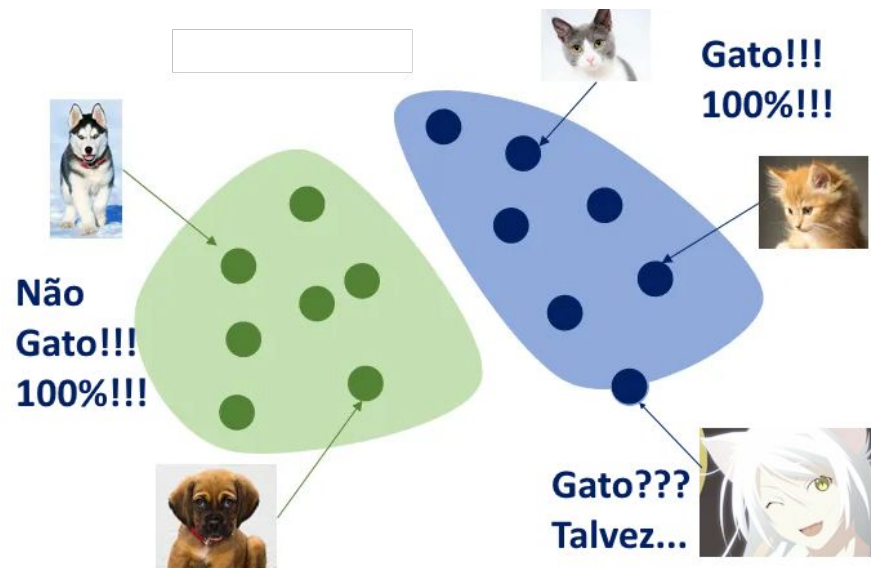
Embeddings de Imagem

- Em um modelo de imagem, os conceitos são uma **distribuição espalhada em uma região do espaço latente!!!!**



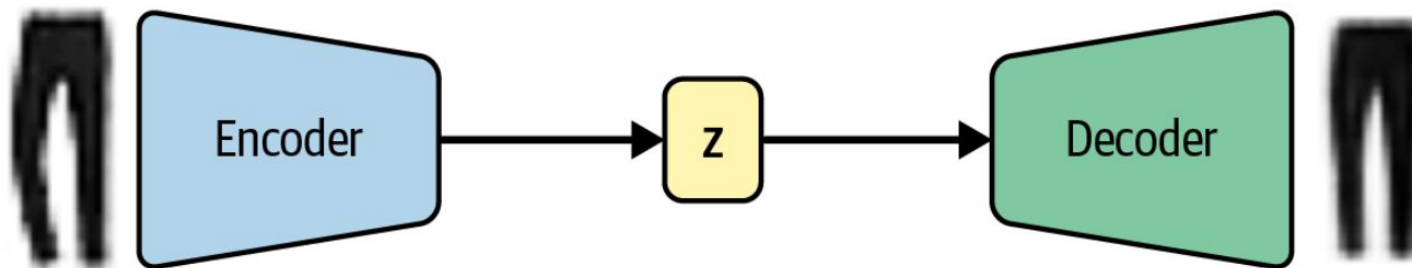
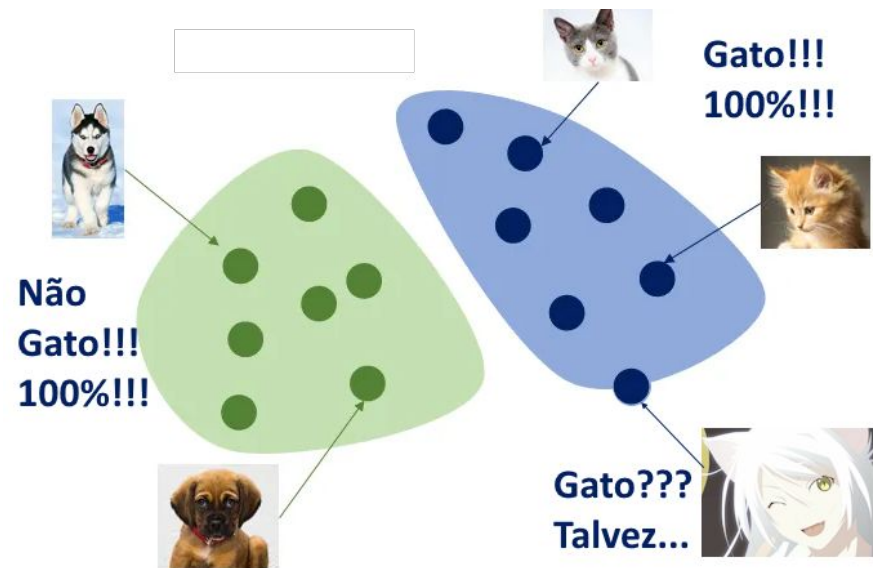
Embeddings de Imagem

- Em um modelo de imagem, os conceitos são uma **distribuição espalhada em uma região do espaço latente!!!!**
- Seja a distribuição de gatos:
 - Gatos mais pertos da média são ideias mais tradicionais do que é um gato!
 - Gatos nem tão gatos assim estão afastados...



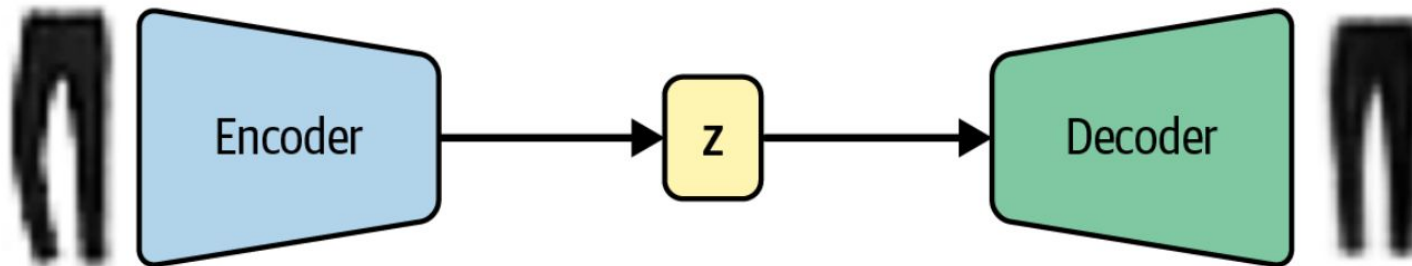
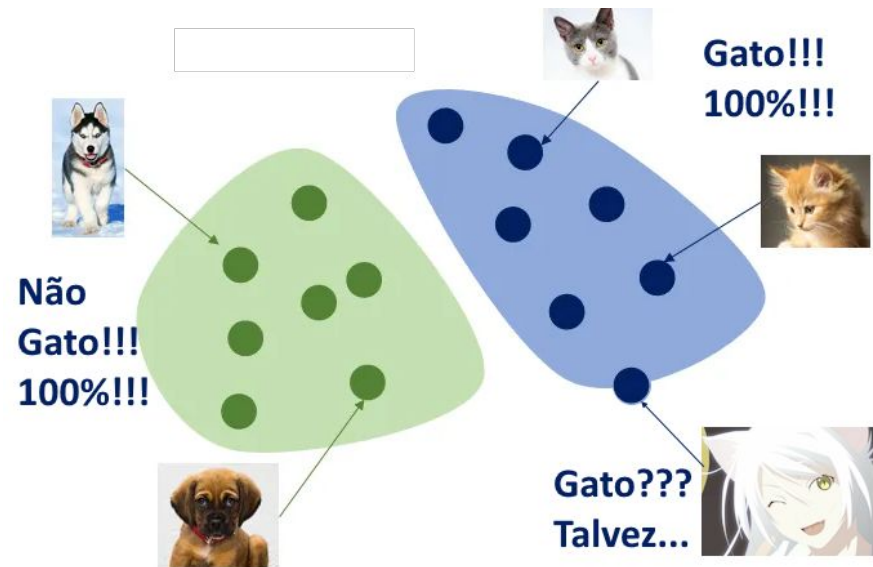
Embeddings de Imagem

- Então, queremos que o embedding em imagens seja um **artifício matemático** que descreva a **probabilidade** do embedding pertencer a determinada **distribuição** de um conceito.



Embeddings de Imagem

- Então, queremos que o embedding em imagens seja um **artifício matemático** que descreva a **probabilidade** do embedding pertencer a determinada **distribuição** de um conceito.
 - Função de probabilidade de densidade.



Embeddings de Imagem

- Função de probabilidade de densidade.
- Embedding = média + $\log(\text{variância}) * N(0,1)$
- Embeddings deixam de virar estáticos e viram estocásticos!!!
- Essa arquitetura se chama **Variational Autoencoder**.

